

CS 660: Data Mining For Decision Making

Lecture 9 (Week 3)

Varun Dutt

School of Computing and Electrical Engineering
School of Humanities and Social Sciences
Indian Institute of Technology Mandi, India

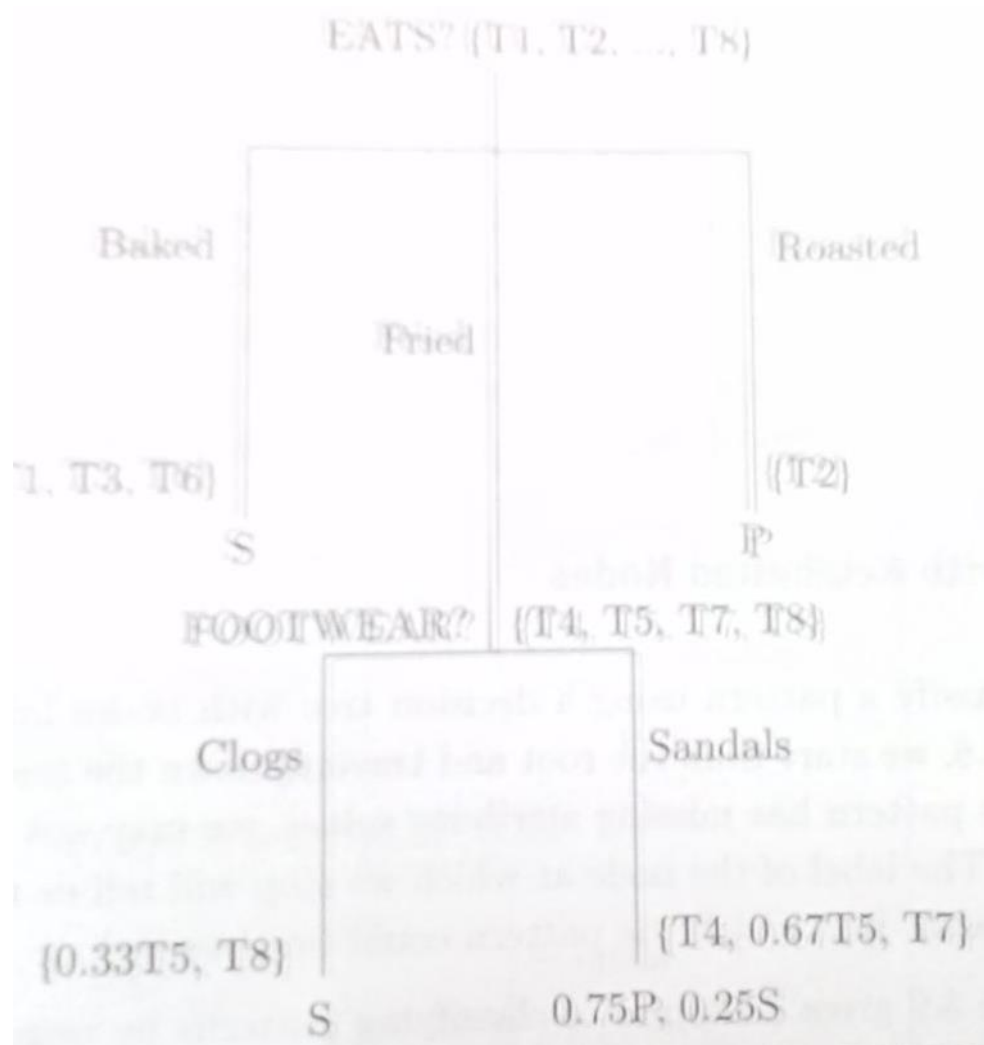


Scaling the Heights

Decision Tree Classifier on Missing Value Attributes

NAME of training pattern	Attributes			Class
	HABIT	EATS	FOOTWEAR	
T1	Gabby	Baked	Clogs	Student
T2	Gabby	Roasted	Sandals	Professor
T3	Gabby	Baked	Sandals	Student
T4	Quiet	Fried	Sandals	Professor
T5	Gabby	Fried	—	Student
T6	Quiet	Baked	Sandals	Student
T7	Gabby	Fried	Sandals	Professor
T8	Quiet	Fried	Clogs	Student

Decision Tree After Considering Attributes



Weights to Leaves

2. The sum of the weights is calculated as

$$\text{sum_of_weights} = \sum_{i=1}^n w_i$$

3. For every training pattern T in V_0 whose value of attribute A is missing, do steps 3.1 and 3.2.

3.1. Let f_0 be the fraction of pattern T present in V_0 .

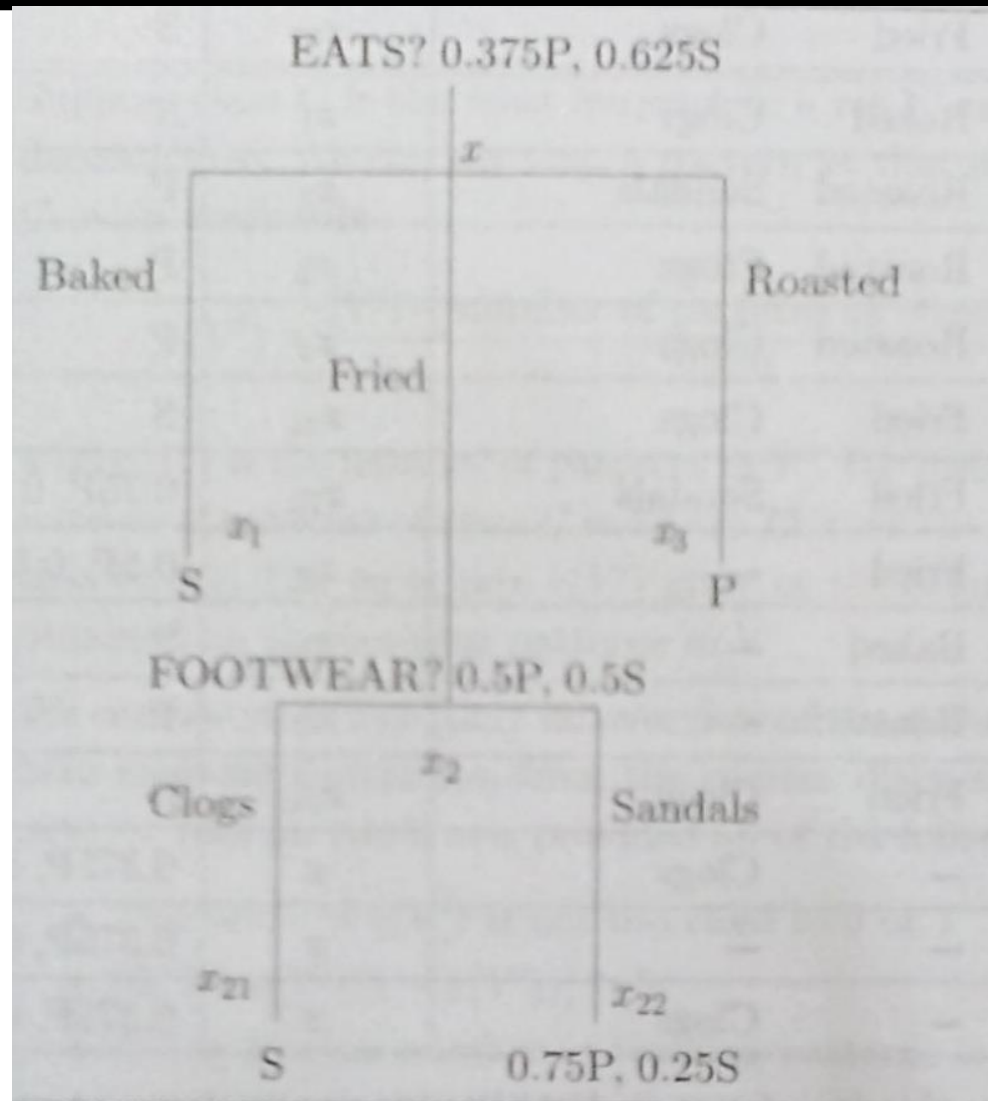
3.2. For $j = 1, 2, \dots, n$ do step 3.2.1.

3.2.1. Add to set V_j the f_j th fraction of pattern T , where

$$f_j = \frac{w_j}{\text{sum_of_weights}} f_0$$

4. Return from the procedure.

Final Decision Tree



Classification of Patterns

T1	Gabby	Baked	Clogs	x_1	S
T2	Gabby	Roasted	Sandals	x_3	P
T3	Gabby	Baked	Sandals	x_1	S
T4*	Quiet	Fried	Sandals	x_{22}	0.75P, 0.25S
T5*	Gabby	Fried	—	x_2	0.5P, 0.5S
T6	Quiet	Baked	Sandals	x_1	S
T7*	Gabby	Fried	Sandals	x_{22}	0.75P, 0.25S
T8	Quiet	Fried	Clogs	x_{21}	S
R1	Quiet	Baked	Clogs	x_1	S
R2	Quiet	Roasted	Sandals	x_3	P
R3	Gabby	Roasted	Clogs	x_3	P
R4	Quiet	Roasted	Clogs	x_3	P
R5	Gabby	Fried	Clogs	x_{21}	S
R6	—	Fried	Sandals	x_{22}	0.75P, 0.25S
R7	—	Fried	—	x_2	0.5P, 0.5S
R8	—	Baked	—	x_1	S
R9	—	Roasted	—	x_3	P
R10	—	Fried	Clogs	x_{21}	S
R11	—	—	Clogs	x	0.375P, 0.625S
R12	Gabby	—	—	x	0.375P, 0.625S
R13	Gabby	—	Clogs	x	0.375P, 0.625S

FIGURE 3.9 Classifications of some professor-student patterns using the decision tree given in Fig. 3.1. A missing value of an attribute is shown by a dash. Patterns T1 to T8 are the training patterns taken from Fig. 3.1. An asterisk besides the name of a training pattern indicates that its classification here is not identical to that in its training set given in Fig. 3.1. Patterns R1 to R13 are recall patterns, of which R1 to R4 are the same as those in Fig. 3.1. The classifications for patterns R11 to R13 have been obtained from the label at the root of the decision tree. For such patterns, an alternative method of classification is discussed in Section 3.2.

Decision Tree Classifier with Numeric Attributes

Table 1.3 Weather Data with Some Numeric Attributes

Outlook	Temperature	Humidity	Windy	Play
Sunny	85	85	false	no
Sunny	80	90	true	no
Overcast	83	86	false	yes
Rainy	70	96	false	yes
Rainy	68	80	false	yes
Rainy	65	70	true	no
Overcast	64	65	true	yes
Sunny	72	95	false	no
Sunny	69	70	false	yes
Rainy	75	80	false	yes
Sunny	75	70	true	yes
Overcast	72	90	true	yes
Overcast	81	75	false	yes
Rainy	71	91	true	no

Theory for Numeric Value Attributes

The method we described in Section 4.3 only works when all the attributes are nominal, whereas, as we have seen, most real datasets contain some numeric attributes. It is not too difficult to extend the algorithm to deal with these. For a numeric attribute we will restrict the possibilities to a two-way, or binary, split. Suppose we use the version of the weather data that has some numeric features (see Table 1.3). Then, when temperature is being considered for the first split, the temperature values involved are

64	65	68	69	70	71	72	75	80	81	83	85
yes	no	yes	yes	yes	no	no	yes	no	yes	yes	no
						yes	yes				

Repeated values have been collapsed together, and there are only 11 possible positions for the breakpoint—8 if the breakpoint is not allowed to separate items of the same class. The information gain for each can be calculated in the usual way. For example, the test *temperature* < 71.5 produces four *yes*'s and two *no*'s, whereas *temperature* > 71.5 produces five *yes*'s and three *no*'s

Theory for Numeric Value Attributes

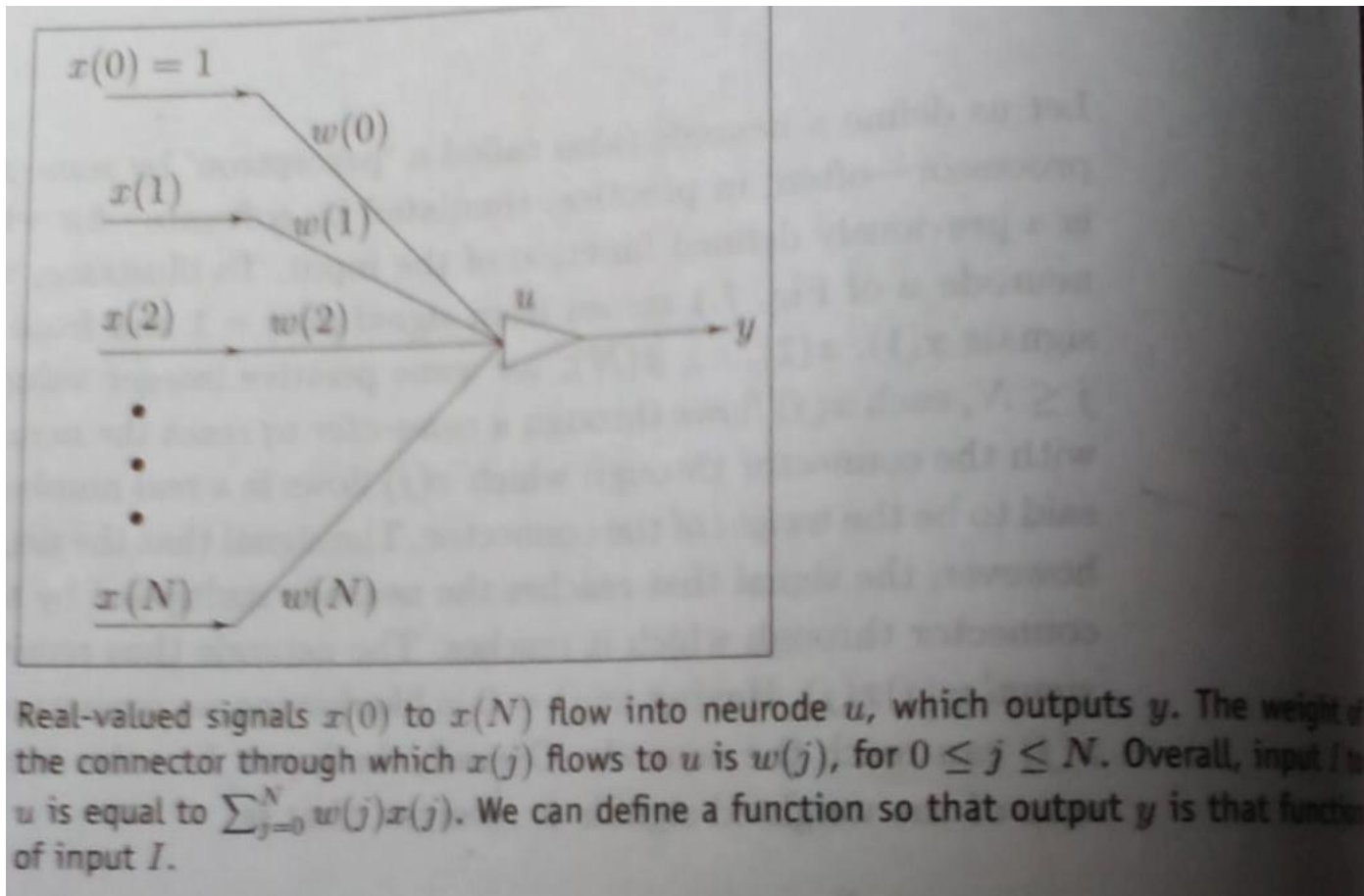
It is common to place numeric thresholds halfway between the values that delimit the boundaries of a concept, although something might be gained by adopting a more sophisticated policy. For example, we will see in the following that although the simplest form of instance-based learning puts the dividing line between concepts in the middle of the space between them, other methods that involve more than just the two nearest examples have been suggested.

Demo on Decision Trees in WEKA

- Student – Professor Training dataset with and without missing attribute values
- Weather data with numeric attributes

Neural Network: Perceptron or Neurode

- Connectionist Approach



Theoretical Equations

$$I = \sum_{j=0}^N w(j)x(j)$$
$$= w(0) + \sum_{j=1}^N w(j)x(j) \quad (\text{because } x(0) = 1)$$

The output y of the neurode can be defined to be some function of the input I , for instance,

$$y = \begin{cases} 0 & \text{if } I < t \\ 1 & \text{otherwise} \end{cases}$$

Later in this chapter, we will see how a

$\Delta w(j) = \text{Beta} * (z - y) * x(j)$, where Beta is the learning rate and z is the desired output

Two kinds of updating: Epoch (after all training patterns) and Case (after each training pattern)

AND GATE via Neural Network

Training pattern	Attributes corresponding to		Class	Desired output z of neuron
	$x(1)$	$x(2)$		
1	1	1	YES-AND	1
2	0	1	NO-AND	0
3	1	0	NO-AND	0
4	0	0	NO-AND	0

Training set for a neuron to model an AND gate. The desired output z of the neuron is different for the two classes, as it should be.

Considering Footwear First

Epoch	$x(1)$	$x(2)$	Weights		$I = w(1)x(1) + w(2)x(2)$	y	z	$\Delta w(j) = \beta(z - y)x(j)$	
			$w(1)$	$w(2)$				$\Delta w(1)$	$\Delta w(2)$
1	1	1	0.1	0.2	0.3	0	1	0.2	0.2
	0	1	0.3	0.4	0.4	0	0	0.0	0.0
	1	0	0.3	0.4	0.3	0	0	0.0	0.0
	0	0	0.3	0.4	0.0	0	0	0.0	0.0
2	1	1	0.3	0.4	0.7	0	1	0.2	0.2
	0	1	0.5	0.6	0.6	0	0	0.0	0.0
	1	0	0.5	0.6	0.5	0	0	0.0	0.0
	0	0	0.5	0.6	0.0	0	0	0.0	0.0
3	1	1	0.5	0.6	1.1	1	1	0.0	0.0
	0	1	0.5	0.6	0.6	0	0	0.0	0.0
	1	0	0.5	0.6	0.5	0	0	0.0	0.0
	0	0	0.5	0.6	0.0	0	0	0.0	0.0

FIGURE 7.3 Training of a neuron to model an AND gate. As shown in the first row of epoch 1, we have $w(1) = 0.1$ and $w(2) = 0.2$ by random initialization. With $x(1) = 1$ and $x(2) = 1$, input $I = w(1)x(1) + w(2)x(2) = (0.1 \times 1) + (0.2 \times 1) = 0.3$. According to the function defined for output in step 4 of the training procedure, since $I < 0.8$, the observed output $y = 0$. The desired output z is, however, 1. Defining learning constant β to be equal to 0.2, we get $\Delta w(1) = \beta(z - y)x(1) = 0.2 \times (1 - 0) \times 1 = 0.2$. Similarly, $\Delta w(2) = \beta(z - y)x(2) = 0.2 \times (1 - 0) \times 1 = 0.2$. Using case updating, each of $w(1)$ and $w(2)$ is increased by 0.2. Thus, $w(1)$ becomes $0.1 + 0.2 = 0.3$, and $w(2)$ becomes $0.2 + 0.2 = 0.4$. This is shown in the second row of epoch 1 in the columns for $w(1)$ and $w(2)$. The training of the neuron proceeds similarly row by row. The training is finished after the third epoch when no changes occur in the weights $w(1)$ and $w(2)$, the difference between the desired output and the observed output having become 0 for each training pattern. The training finishes with $w(1) = 0.5$ and $w(2) = 0.6$. The trained neuron is shown in Fig. 7.4.

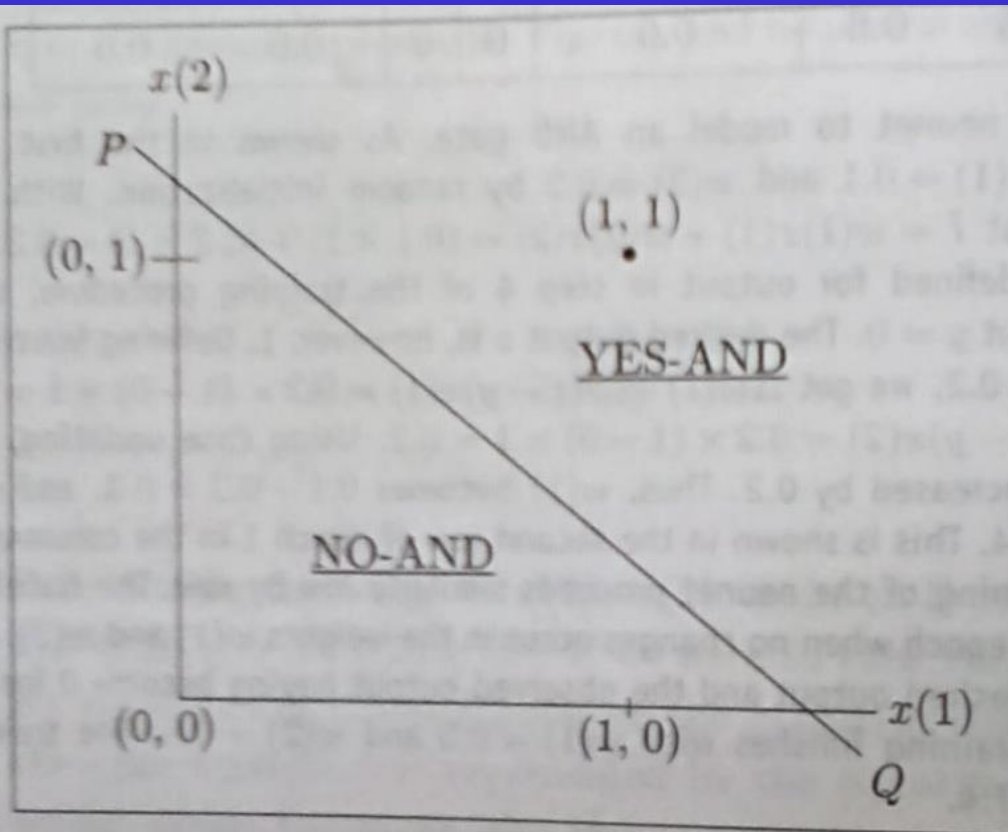
$$w(1) = 0.5$$

$$w(2) = 0.6$$

$$I = w(1)x(1) + w(2)x(2)$$

$$y = 1 \text{ if } I \geq 0.8$$

Linear Classification



Representing the AND gate model on the $x(1)$ - $x(2)$ coordinate plane. A straight line⁹ divides the four points into two regions corresponding to the two classes YES-AND and NO-AND.

Analytical Argument

$$y = \begin{cases} 0 & \text{if } I < t \\ 1 & \text{otherwise} \end{cases}$$

For the first pattern in the training set given in Fig. 7.2, output y should be 1; for that to happen, input I should be greater than or equal to t , that is,

$$[w(1)x(1) + w(2)x(2)] \geq t \quad (7.5)$$

since $I = w(1)x(1) + w(2)x(2)$. From the second to the fourth patterns of Fig. 7.2, output y should be 0, for which, input I should be less than t , that is,

$$[w(1)x(1) + w(2)x(2)] < t \quad (7.6)$$

Substituting the values of $x(1)$ and $x(2)$ in Eqns (7.5) and (7.6), we obtain the following four equations:

$$[w(1) \times 1 + w(2) \times 1] \geq t \text{ (from the first pattern)} \quad (7.7)$$

$$[w(1) \times 0 + w(2) \times 1] < t \text{ (from the second pattern)} \quad (7.8)$$

$$[w(1) \times 1 + w(2) \times 0] < t \text{ (from the third pattern)} \quad (7.9)$$

$$[w(1) \times 0 + w(2) \times 0] < t \text{ (from the fourth pattern)} \quad (7.10)$$

Simplifying Eqns (7.7) to (7.9), we get the following constraints on the weights $w(1)$ and $w(2)$ for a neuron to model an AND gate

$$[w(1) + w(2)] \geq t \text{ [from Eqn (7.7)]}$$

$$w(2) < t \text{ [from Eqn (7.8)]}$$

$$w(1) < t \text{ [from Eqn (7.9)]}$$

Figure 7.3 shows that, with $t = 0.8$, the neuron finished its training at $w(1) = 0.5$ and $w(2) = 0.6$. Clearly, the weights satisfy the above constraints.

Things to do

- Read Decision Tree Classifiers from Shinghal, 2011 (Material posted on Moodle)
- Read WEKA from Witten's book
- Do Assignment 2



Questions or Comments?